

Exact Top-K Selection on a Dataflow Many-Core: A Measured Design-Space Study

Anonymous Authors
Paper #XXX — IPDPS 2027 submission

Abstract—Exact top- k selection sits on the decode-time critical path of large language model serving: sampling selects $k=32$ –64 winners from a padded vocabulary shard every token, sparse-attention indexers select $k=512$ –2048 keys per layer, and mixture-of-experts gates fire every forward pass. GPU practice treats the problem as solved—count, partition, scatter, repeat—while accelerator studies either approximate the selection or never measure it. We present a measured design-space study of exact top- k on a Tenstorrent Blackhole p150a, a dataflow many-core with 130 Tensix workers on a 13×10 mesh NoC, software-managed SRAM, and neither atomics nor scatter in the compute path. Single-ingredient silicon microbenchmarks show why GPU radix-select economics do not transfer: exact SFPU counting yields 1 bit per 2.0 cycles/vector, a count-to-decision rendezvous costs 81 cycles, and materialization—not counting—is the load-bearing gap (dense emission 13.0 cycles/element against a 0.5 bar). The design space instead rewards parallelizing the data-oblivious comparison network: a column-parallel log-tree operator whose bitonic leaves feed a semaphore-tree merge, governed by a validated cost model $2\lceil C/P \rceil + \lceil \log_2 P \rceil$. The operator measures $34.3\mu\text{s}$ at $k=2048$, $N=65,536$ (bf16) on 32 cores— $18,401\times$ over the pre-study stock path—and scales monotonically to $23.3\mu\text{s}$ at $k=512$, $N=262,144$ on 104 cores. A distribution-free chunk-skip law $P(\text{skip}) \approx e^{-K/(c+1)}$ with a compile-time $K/4$ gate adds $1.82\times$ (-45.1%) at $k=32$, $N=65,536$. End to end, the user-facing `ttnn.topk` call at $k=2048$, $N=65,536$ improves from 631.5ms to $63.4\mu\text{s}$ — $9,956\times$ —and the surviving gap to the vendor roofline spans 9.7 – $29.9\times$ across all 24 measured cells. The study is honest about its own dynamics: the incumbent bitonic engine absorbed the study’s routing and placement fixes and outran the radix-select model it was measured against. We also report the first public characterization of the Tensix datapath’s numeric semantics: bf16 canonicalization, a native sign-magnitude total order including NaN, and a 1-in-8-sampled, 32-bin-aliased packer exponent histogram.

Index Terms—top- k selection, dataflow architectures, network-on-chip, many-core, Tenstorrent Blackhole, performance characterization

I. INTRODUCTION

On a Tenstorrent Blackhole p150a, Qwen3 decode sampling under tensor parallelism 4 shards a 151,936-entry vocabulary into 37,984 logits per device and pads the shard to 65,536—the one power-of-two width that fails the shipping bitonic engine’s $W < 65,535$ routing gate. The call lands on a single-core linear factory that walks logits at $\approx 137\text{ns/element}$: $9,596\mu\text{s}$ per sampled token for a $k=32$ selection, while the other 129 cores idle. The cliff is not a corner case. At the sparse-attention indexer shape $k=2048$, $N=65,536$ (bf16), the stock `ttnn.topk` path measured 631.5ms per call at the start of this study; the same call on the same silicon now measures $63.4\mu\text{s}$ — $9,956\times$ —with no new hardware. This paper argues

one sentence: exact top- k on a commercial NoC-mesh dataflow chip sat four orders of magnitude off its achievable floor, and a measured design-space study—not a clever new algorithm—closed the gap.

Blackhole is a dataflow many-core: 130 Tensix workers on a 13×10 grid, each with five RISC-V processors, a 32-lane SIMD unit (SFPU), and software-managed SRAM, joined by a 2D NoC [1]. Every fast GPU top- k mechanism leans on primitives this machine lacks — atomic histograms, scatter compaction, grid-wide synchronization [2], [3], [4]; cores synchronize through NoC semaphores (§II-A).

The missing primitives are not one vendor’s oversight. AI accelerators across the industry traded coherent shared memory and atomics for mesh bandwidth and software-managed SRAM, because matrix multiplication rewards the trade. Selection is the canonical data-dependent workload the trade punishes, and LLM decoding keeps placing it on the critical path (§II-A fixes the data contract) [5], [6], [7], [8]. Nobody has measured what the trade costs here: no sorting or selection publication exists for the dataflow-accelerator class (§VII), and TPU studies escape into approximation [9], [10]. Mesh-selection theory proved step bounds on abstract arrays [11], [12], but returned neither values with indices nor a measurement on real silicon.

We measured the design space ingredient by ingredient instead of betting on one kernel. The study decomposes every exact top- k engine along three questions—winner identification, data touched per pass, decision cost between passes—and holds Blackhole against each with single-ingredient microbenchmarks (§III). The answers overturn the GPU playbook: counting is narrow, decisions are affordable, and materialization is the wall, so radix select degenerates to threshold bisection and loses. The space instead rewards parallelizing the data-oblivious comparison network as a column-parallel log-tree operator (§IV); the one data-dependent mechanism that survives without scatter is a chunk-skip rule, sound by construction, that obeys a distribution-free rank-statistic law and gates itself at compile time.

The study is candid about its own dynamics. The incumbent bitonic engine absorbed the study’s routing and placement fixes and finished at $34.4\mu\text{s}$ on the anchor cell ($k=2048$, $N=65,536$)—inside the uncertainty band of the radix-select model priced from our own microbenchmarks; a pre-registered stop rule closed that track (§III). The surviving gap to the vendor roofline is 9.7 – $29.9\times$ across all 24 measured cells (§VI).

This paper contributes:

- **Quantification**, via single-ingredient silicon microbenchmarks, of why GPU radix-select economics do not transfer to a NoC-mesh dataflow core: exact SFPU counting is 1 bit per 2.0 cycles/vector, decisions cost 81 cycles, and materialization—not counting—is the load-bearing gap, with dense emission at 13.0 cycles/element against a 0.5 bar (§III).
- **Design and measurement** of a column-parallel log-tree top- k operator on the 130-core Blackhole mesh—semaphore-tree rendezvous without atomics or global synchronization—with the validated cost model $2\lceil C/P \rceil + \lceil \log_2 P \rceil$ and a cost-optimal rectangle embedding: 34.3 μ s at $k=2048$, $N=65,536$ on 32 cores ($18,401\times$ over the pre-study stock `ttnn.topk`) and monotone scaling to 23.3 μ s at $k=512$, $N=262,144$ on 104 cores (§IV, §VI). To our knowledge, the first *measured* full top- k (values and indices) on commercial NoC-mesh silicon (§VII).
- **Derivation and silicon validation** of a distribution-free chunk-skip law $P(\text{skip}) \approx e^{-K/(c+1)}$ for streaming bitonic cascades, with a soundness proof, a compile-time $K/4$ gate, and a calibrated pre-build forecast that eliminated the losing variant; the surviving variant measures $1.82\times$ (-45.1%) at rows=2, $k=32$, $N=65,536$ (§IV).
- **First public characterization** of the Tensix compute datapath’s numeric semantics and sorting-relevant primitive costs: bf16 canonicalization (NaN $\rightarrow$ Inf, $-0 \rightarrow +0$, subnormal $\rightarrow +0$), a native sign-magnitude total order including NaN, a 1-in-8-sampled and 32-bin-aliased packer exponent histogram, and measured synchronization floors (§V)—absent from all prior Tensix studies (§VII).

Every performance cell behind these claims is gated bit-exact against a host golden—never a similarity threshold—and every headline number traces to a committed measurement artifact; the ledger and canonical sweep harness will be released, and §VIII discloses the LLM-agent campaign methodology.

II. BACKGROUND

A. Problem Statement and Data Contract

The object of study is exact top- k selection: given one or more rows $x \in \text{bf16}^N$ resident in device DRAM, return the k largest elements of each row — values *and original indices*, exact rather than approximate, sorted descending. Under `stable=false`, ties may resolve to any valid winner set but must be deterministic for a fixed input. A 62-cell differential contract suite pins what the shipping engines actually promise: the returned values are the exact top- k multiset of the input *as canonicalized by the datapath* (bf16 specials are rewritten on entry; Section V pins the rules) under the hardware’s native sign-magnitude total order; every index is unique and satisfies $x[\text{index}] = \text{value}$ bit-exactly (rows with fewer than k finite values emit a documented sentinel index); repeated launches return bit-identical outputs. bf16 is the datatype at every surveyed production call site; fp32 traverses bit-exactly.

Production adds three contract variants: indices-only versus (values, indices) outputs, multi-row batches, and a valid-length prefix that masks a stale tail.

Where does x come from, and where do the results go? The row is always the output of the operator immediately upstream — an lm-head matmul, an attention-indexer score, or an expert-gate projection — written to a DRAM-interleaved tensor in the producer’s layout; results return to DRAM for the consumer. Three consumer classes cover every surveyed call site (Table IV): token sampling needs values and indices at $k=32$ over vocabulary shards of $N=16\text{--}65\text{K}$ every decoded token; sparse-attention KV gather needs indices only at $k=512\text{--}2048$ over contexts up to 1M, every layer; mixture-of-experts dispatch needs $k=4\text{--}16$ winners over 128–512 experts every layer. A selection engine’s real cost therefore includes the layout conversions between the producer’s format, the engine’s, and the consumer’s — which is why Section VI prices end-to-end composites, never bare kernels.

The machine class sets the difficulty. CPU selection — vectorized quickselect of the `x86-simd-sort/vqsort` family [13], [14] — rides coherent caches and branch predictors: branchy, data-dependent control flow is cheap and the scalar pivot logic is free. GPU selection rides shared-memory atomic histograms, scatter compaction, and grid-wide synchronization (§II-D). Blackhole has none of these: no cross-core atomics, no scatter or compaction in the compute path, no global barrier — synchronization is NoC semaphores over software-managed SRAM, the 32-lane SFPU counts 1–3 bits per pass, and the fast path is a fixed, data-oblivious instruction stream, so every data-dependent decision costs a measured 81-cycle rendezvous. Every verdict in Section III and every design choice in Section IV traces back to this paragraph.

B. Blackhole and the Tensix Core

The Tenstorrent Blackhole p150a exposes 130 programmable Tensix workers on a 13×10 grid, joined by a 2D network-on-chip (NoC) [1]; the count is a property of this unit (harvesting varies per part), and no result here hardcodes it. Each core owns $\approx 1.5\text{MB}$ of software-managed L1 SRAM [1]; there is no cache hierarchy and no coherence protocol.

Five RISC-V processors share that L1: two move data over the NoC (one reads, one writes), and three compute as an `unpack $\rightarrow$ math $\rightarrow$ pack` pipeline. The math stage owns the SFPU, a 32-lane SIMD engine, and stages its output in the Dst register file that pack consumes. An operator is therefore three-to-five cooperating programs per core, synchronized through producer/consumer circular buffers (CBs) in L1. Sections III and V price these units one at a time; Section IV spends them.

C. The Incumbent Bitonic Engines

The vendor stack routes `ttnn.topk` through a bitonic local sort in the low-level kernel library (`topk_local_sort`) [15], [16]. The bitonic leaf sorts a window of M elements resident in Dst, where M snaps upward from the requested k : $k \leq 512$ gives $M=512$, $k \leq 1024$ gives 1024, and larger k gives 2048. We fix the

paper’s notation here: a row holds N elements, split into $C = N/M$ chunks; P cores cooperate on one row; K is the user’s k . All later sections inherit these symbols.

Before this study, two engines shipped. A multi-core bitonic path accepts only rows with power-of-two width, $W < 65,535$, $k \leq 64$, and $N \geq 4096$. A single-core linear factory catches everything else at ≈ 137 ns/element: $695 \mu\text{s}$ at $W=5,000$ ($k=32$), growing to 17.95 ms at $W=65,536$ ($k=64$). Between the two engines sits a measured cliff. At the $W < 65,535$ gate, a $k \leq 32$ row jumps from $106.7 \mu\text{s}$ (multi-core, 65 cores, $N=8,192$) to 9.49 ms (one core, $N=65,534$) — $\approx 89\times$ more time for $8\times$ more elements. We state the cliff here as architecture context; Section VI owns the fix. The bitonic engine itself is data-oblivious — an input-independent instruction stream, no decisions, no shrinking candidate sets — one corner of the design space Section III maps and prices.

D. What GPU Selection Assumes

Exact top- k on GPUs is a mature family; each branch leans on a specific hardware affordance — the import price Blackhole must pay to adopt it. **Radix/bucket select** builds 8–12-bit exact histograms in shared memory via atomics, scatters survivors to a compacted buffer, and recurses [2], [17], [4]: wide exact counting plus cheap materialization of a shrinking candidate set. **WarpSelect** kills most elements at a single compare against a per-lane running threshold and parks survivors in per-lane queues [18]: a cheap lane-local compare plus a place to park survivors (Section IV contrasts this element-granular threshold with our chunk-granular skip). **Threshold-guess families** predict the k -th value from the previous decode step, count against the guess, and refine until the boundary is exact [7]: cheap counting plus a cheap count-to-decision rendezvous. **The escape hatch:** TPU-KNN and its two-stage successors avoid exact selection outright, reshaping top- k into matmul-friendly partial reductions [9], [10] — the accelerator industry’s confession that exact selection is hostile to matmul-centric machines. This paper measures the exact path anyway: Section III prices each affordance above on silicon.

III. WHY GPU SELECTION ECONOMICS DO NOT TRANSFER

GPU radix select wins because its hardware sells wide exact counting and cheap materialization [2], [3], [17], [4]; Blackhole quotes different prices. This section decomposes exact selection into three orthogonal questions, prices each ingredient on silicon, and locates the ingredient that kills the family — materialization, not counting and not synchronization. The (A)–(F) vocabulary defined here carries through Sections IV–IX.

A. Three Questions, Six Answers

Every exact top- k engine answers three independent questions. **Q1 — winner identification:** either (A) a comparison network — a fixed, data-oblivious schedule of compare-exchanges (bitonic sort/merge, WarpSelect [18]) —

TABLE I
GPU RADIX-SELECT INGREDIENTS, PRICED ON BLACKHOLE.

Ingredient	GPU form	Blackhole, measured	Verdict
Wide exact counting (Q1-B)	8–12 bits/pass, shared-mem. atomics [2], [4]	SFPU: 1 bit @ 2.0 cyc/vec, 3 bits @ 3.0; additive on the 3.94 cyc/vec unpack floor (+2.42/+4.52 in-stream; 2.44/4.53 isolated) [cgtceq]. RISC: 256-bin histogram 7.56–7.90 cyc/elem vs. 3.02 load floor; 8.23 with 4-way sub-histograms; L1-resident 15.5–19.9 [risc-scan]	narrow (SFPU) or slow (RISC) — degenerates to bisection
Cheap decisions (Q3-F)	grid sync / fused kernels	81 cyc/decision best (sync + fold + MMIO read); semaphore 101, sentinel 98; no fold 756–773; MMIO ≈ 10 cyc/word [cgtceq]	affordable — not the blocker
Materialization (Q2-D)	scatter / compaction	dense RISC emit 12.97–17.81 cyc/elem vs. a ≈ 0.5 bar; SFPU has no scatter; packer compressed stream 0.63 cyc/orig-elem, consumer-only [risc-scan]	the load-bearing gap
A slow incumbent	thrust::sort	34.4 μs ($k=2048$, $N=65,536$); 23.3 μs ($k=512$, $N=262,144$) after this study’s own fixes [sweep]	bar moved —15–27% during the study

or (B) counting/partitioning, which counts elements beyond a boundary and *decides* where to look next. **Q2 — data touched per pass:** either (C) full-data passes every time, or (D) shrinking candidate sets, by scatter compaction or count-guided skipping. **Q3 — decision cost between passes:** (E) nothing, for oblivious schedules; or (F) a synchronization rendezvous that carries the count to a scalar decision-maker and redistributes the verdict.

GPU radix select answers (B)+(D)+cheap-(F): 8–12-bit atomic histograms, scatter compaction, grid-sync decisions (Section II) [2], [4]; the shipping Blackhole engine answers (A)+(C)+(E) — ≈ 2 cycles/element on every pass, zero decisions. Can Blackhole move to (B)?

B. The Ingredients, Measured

Table I prices the four ingredients, each cell tagged with its benchmark; all costs are slope-measured cycles, gated bit-exactly against host goldens (§VI-A).

Wide exact counting (Q1-B). The 32-lane SFPU counts exactly but narrowly: one predicate bit per pass at 2.0000 cycles/vector, three bits at 3.0, riding additively on the 3.94 cyc/vec unpack floor. A GPU pass resolves 8–12 bits [2], [4]; an SFPU pass resolves 1–3, so a radix engine on this datapath collapses into a bisection. The scalar cores are no rescue: a 256-bin RISC histogram runs 7.56–7.90 cycles/element against a 3.02 pure-load floor.

Cheap decisions (Q3-F). Decisions measure 81 cycles each at best: a `tensix_sync`, a cross-lane SFPU fold, and one memory-mapped read of the DST register file (folding is mandatory — raw-lane reads cost 756–773 cycles, $\approx 9\times$ worse). At 81 cycles ≈ 60 ns¹ and ≈ 14 decisions per row (Section III-C), decisions are affordable — not the blocker.

¹Cycle→time conversions assume a 1.35 GHz busy clock; the clock under load has not been captured (idle 800 MHz measured). Section VI states the caveat once for the whole paper.

Materialization (Q2-D). Dense survivor emission on a RISC core measures 12.97–17.81 cycles/element against a ≈ 0.5 cycles/element bar for compaction to pay — dead. The SFPU has no scatter path at all. The sole survivor, the packer’s zero-compressed output stream with a skip-zero RISC consumer, costs 0.63 cycles per original element: $\approx 3\times$ under one bitonic leaf pass but $2\text{--}5\times$ above GPU-paper compaction — and it is consumer-only, measured on uniform input: no device-side decoder exists, and the composed producer+consumer cost was never measured — reopening condition #1 in Section IX.

C. Degeneration to Threshold Bisection

Without (D), every counting pass is strictly additive to the bitonic finish that must still run; what survives of (B) is threshold bisection on the key space, monotone because sign-magnitude keys order the probes (Section V). Over 57 input cells — random, clustered, and all-equal distributions across seeds, a $K \in \{31, 32, 33\}$ straddle, ties, and $\pm 0/\text{Inf}/\text{NaN}/\text{denormal}$ specials — bisection to the exact K -th threshold costs a median of 14 decisions and 2,313 cycles per 1,024-element row, worst case 17 decisions on clustered, all-equal, and tied inputs, exactly where key-space bisection predicts; the certification invariant $C_{\text{gt}} < K \leq C_{\text{gt}} + C_{\text{eq}}$ held field-exactly on every row. Per decision, ≈ 165 cycles — one count pass ($\approx 64\text{--}70$), one rendezvous (81), loop overhead: the (F) matrix composes as priced. At $\approx 1.7\mu\text{s}$ per row the exact arm is honest — Floyd–Rivest economics [19], not radix economics. The family fits Blackhole only in its degenerate corner, (B)+(C)+(F).

D. Why the Track Closed

A selector composed from the measured components — packer pre-filter, count passes, rendezvous, materialization, final K -sort — prices at $\approx 21\text{--}30\mu\text{s}$ per large cell. The pre-registered stop rule, written before any measurement, demanded an incumbent $\geq 2\times$ the model to justify building. The incumbent finished the study at $34.4\mu\text{s}$ ($k=2048$, $N=65,536$) — inside the model’s uncertainty band — because the study’s own first gate produced two structural fixes (a small- k routing repair and a core-count cap raise, Sections IV and VI) that made the incumbent 15–27% faster while the selector was being priced. A selector that must outrun its own evaluation’s side effects is not a production bet; the track closed by its own rule. The machinery was banked: the rendezvous and fold primitives price Section IV’s chunk-skip decision; the floors join Section V.

IV. A LOG-TREE TOP-K ENGINE FOR A NOC MESH

The map of Section III leaves two live moves. First, parallelize the data-oblivious corner (A)+(C)+(E): split the row across cores and pay a log-depth rendezvous instead of a shared-memory partition. Second, import the shrinking axis (D) in the only form that survives without scatter: skip work, never move data. This section builds both: Sections IV-A–IV-B the column-parallel log-tree operator

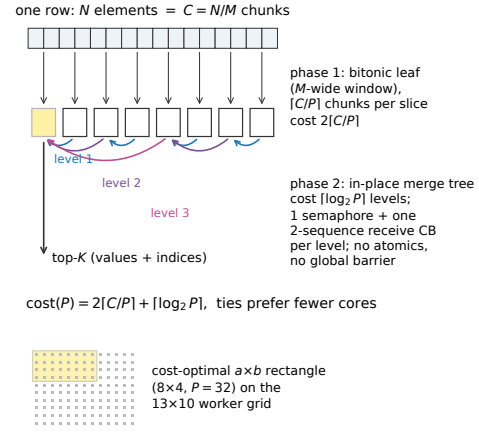


Fig. 1. Column-parallel log-tree top- k operator.

(topk_large_indices), Sections IV-C–IV-E chunk-skip and the calibrated forecast that eliminated its losing variant pre-build.

A. Operator Structure

Figure 1 shows the operator on one row of N elements, using the notation of Section II: the leaf window M snaps to the LLK sort width (§II-C), and the row splits into $C = N/M$ chunks. The program places P slices on a rectangle of cores; each slice streams $\lceil C/P \rceil$ chunks through a bitonic leaf that sorts the arriving chunk, merges it into a resident M -wide descending window, and rebuilds the window order. A $\lceil \log_2 P \rceil$ -level in-place merge tree — the mesh-shaped sibling of FPGA merge-tree sorters [20], applied to selection — then folds the P windows to a root core, which emits the top- K values and indices; every level halves the live cores and merges two M -wide sequences.

The synchronization fabric is deliberately austere. Tree levels map one-to-one onto NoC semaphore levels; each core owns a single receive circular buffer sized for two M -wide sequences; the writer kernels extend to seven partner slots, enough for $P=128$. The operator uses no atomics, no global barrier, and no dedicated DRAM scratch (§II-A) [2], [4]. The tree replaced a serial merge-chain ancestor that folded slices one at a time: at fixed core counts the swap cut $k=2048$, $N=65,536$ from $69.6\mu\text{s}$ to $41.9\mu\text{s}$ ($1.66\times$) and $k=512$, $N=262,144$ from $64.2\mu\text{s}$ to $32.0\mu\text{s}$ ($2.01\times$), and it made runtime monotone in P — the property the cost model below then exploits.

B. Cost Model and Rectangle Embedding

The factory chooses P by minimizing a two-term model in leaf merge units:

$$\text{cost}(P) = 2\lceil C/P \rceil + \lceil \log_2 P \rceil, \quad P \in [2, \min(C, 128)], \quad (1)$$

with ties resolved toward fewer cores. The factor of two prices the leaf honestly: every chunk after the first costs ≈ 2 merge units (sort, merge, rebuild) while chunk 0 costs one. The

unit constant is measured, not assumed: $1.46\mu\text{s}$ per merge unit at $M=512$ and $5.51\mu\text{s}$ at $M=2048$ ($\approx 1,966$ and $\approx 7,440$ cycles²). The leaf term is mesh-selection theory’s multi-packet regime [11], [12], [21] with measured constants; the log term is the mesh’s rendezvous price.

Minimizing Eq. 1 is only half the placement problem; the winner must also embed in the 13×10 worker grid as a contiguous $a\times b$ rectangle. An earlier full-rows-only fit silently clamped P : at $k=2048$, $N=65,536$ it settled for $13\times 2 = 26$ cores while an $8\times 4 = 32$ -core rectangle sat free. Searching all cost-optimal rectangles instead wins 8–27% at the op level across the re-pinned cells: $41.9\rightarrow 34.4\mu\text{s}$ ($26\rightarrow 32$ cores), $58.6\rightarrow 49.6\mu\text{s}$ ($52\rightarrow 64$), $32.0\rightarrow 23.3\mu\text{s}$ ($52\rightarrow 104$), and $15.0\rightarrow 13.1\mu\text{s}$ ($52\rightarrow 64$). On a harvested mesh the embedding search is part of the cost model, not a detail below it. Section VI-C defends the model against the measured 15-point P -sweep, including the ceiling-term bump it predicts.

C. Chunk-Skip: Mechanism and Soundness

Ingredient (D) — shrinking the touched data — survives on this machine only if nothing has to move. Chunk-skip is that import: a per-chunk predicate that decides whether the leaf pipeline needs to run at all. Let T be the running K -th largest survivor, read from the resident sorted window. The rule is strict: skip chunk c iff $\max(\text{chunk}_c) < T$. Chunk 0 always seeds the window; T is monotone nondecreasing because merges only improve the window; boundary ties ($\max = T$) are never skipped; and the decision is a pure function of the input, so the output is deterministic per input.

Soundness admits a short argument: at most $K-1$ elements ever exceed the row’s final K -th largest value v_K , so $T \leq v_K$ at all times; any member x of an exact top- K set satisfies $x \geq v_K \geq T$, so the strict test never fires on x ’s chunk, and x , sitting within the top- $K \leq \text{top-}M$, survives every M -wide merge to the root. The released kernel source carries the full 288-line proof as a header comment. The contrast with the GPU running-threshold family is threefold: WarpSelect thresholds per element, with no analytic skip law [18]; GVR guesses its threshold from the previous decode step and must count, verify, and refine after a misprediction [7]; chunk-skip prunes at M -element granularity, sound by construction — no verification pass exists to pay for — with a distribution-free law and compile-time gate (§IV-D) and a pre-build forecast (§IV-E) that neither GPU mechanism has. Adversarial validation backs the proof: a 36-check battery (all-equal, ascending, last-chunk winners, boundary ties replicated across 64 chunks) is bit-exact in both gated and ungated configurations, and a 20-launch hang battery passed twice.

D. The Skip Law and Its Compile-Time Gate

How often the rule fires is a rank statistic, not a distribution property. For i.i.d. inputs, chunk $c+1$ is skippable iff the top- K

of the first $(c+1)M$ elements all lie in the first cM :

$$\Pr[\text{skip at } c] = \binom{cM}{K} / \binom{(c+1)M}{K} \approx e^{-K/(c+1)}, \quad (2)$$

identical for normal, uniform, and softmax-shaped inputs because only ranks enter. The law is unforgiving at short streams: a conservative $K=512$ threshold skips with probability 2×10^{-307} at $c=1$ and reaches only 1.8% at $c=128$, while an aggressive user- $K=32$ threshold — sound by the argument of Section IV-C even though the resident window is M -wide — reaches 37% at $c=32$ and 78% at $c=128$.

The law hands us the gate. Below $c = K/4$ the skip probability is under $e^{-4} \approx 1.8\%$, so the compile-time gate sets the first tested chunk to $\max(2, K/4)$ — a derivation from Eq. 2, not a tuned constant (the floor of two: the threshold address needs the post-rebuild window layout of Section V, which exists only after the first merge). The gate zeroes the measured ungated overhead (+6.8% at $K=512$ over 128 chunks, +1.8% at $K=1536$ over 25 chunks) while forfeiting under one expected skip per row. The decision itself is cheap: ≈ 150 ns per tested chunk at $M=512$ and ≈ 350 ns at $M=2048$, derived from the ungated A/B (126 tested chunks cost +18.9 μs) and consistent with the 81-cycle rendezvous plus fold components of Section III (footnote 2) — against $\approx 2.0\mu\text{s}$ and $\approx 5.4\mu\text{s}$ saved per skipped chunk.

E. Forecast Before Build: The Variant We Did Not Build

Before any kernel was written, an exact host-side simulation — bf16 sign-magnitude streaming, stimulus matched to the sweep harness, calibrated to reproduce all five pinned measured cells to $\pm 0.0\mu\text{s}$ by construction — priced chunk-skip inside the column-parallel tree. The verdict was a no-go with the weight of a result: 0.00% skip on every realistic distribution on every cell, a predicted +0.1–0.4 μs pure regression from the test overhead. The cause is structural: *the rectangle split and the skip law fight over the same resource, stream length c* — the cost-minimizing split caps each slice at $\lceil C/P \rceil = 1-5$ chunks while Eq. 2 needs $c \gtrsim K/4$. The split already won.

The build therefore went where the law points: the row-parallel path, in which each core streams all C chunks of its own row — long streams, small K , exactly the shapes the stock small- k routing ships to this path. The forecast’s redirect predicted 54.3% skip at $K=32$ on a 128-chunk stream. At rows=2, $K=32$, $N=65,536$ the measured time falls $1.82\times$ (−45.1%), and at rows=8 by $1.57\times$ (−36.3%) — both on 130-core programs whose active cores number $\min(\text{rows}, 130)$, so the rows=2 cell exercises two cores, not the grid. Cells where the law forbids a win are protected by the gate, and the column-parallel guard cell holds with byte-identical tree kernels (Section VI-D shows the A/B bars, quotes the guard numbers, and carries the epistemic caveat). The forecast said no; we did not build that variant — Section VIII returns to forecast-before-build as methodology.

²Cycle forms of Tracy-measured times assume a 1.35 GHz busy clock; the clock under load was not captured (idle 800 MHz measured). Section VI states the policy; all cycle→time conversions in this section carry it.

V. CHARACTERIZING THE TENSIX DATAPATH FOR SELECTION

Selection exercises corners of a datapath that matmul studies never touch: NaN ordering, signed zeros, compare-exchange operand routing, and the cost of getting one count off the SIMD unit. This section banks those facts as an archival reference in the microbenchmark-dissection genre of Jia et al. [22]; no prior Tensix study, including the general Blackhole microbenchmark report [23], reports datapath numerics or selection-primitive costs (§VII). Table II anchors the section. Every cycle figure is a two-point slope measurement with a $2.000\times$ control tripwire (§VI-A), quoted at the precision the harness earned; the section stays in cycles, so no busy-clock conversion caveat applies.

A. Numeric semantics

An identity-op probe shows the bf16 compute datapath rewrites specials before any operation touches them — Table II, row 1, gives the full rules — while index positions survive and fp32 passes bit-exactly. Vendor ISA documentation corroborates the Dst behavior [24]. This rule is the precondition for every correctness gate in the paper (§II-A): every reference model in §VI canonicalizes before comparing bit-exactly.

A failing differential test on $\pm$ NaN specials exposed the second law: the comparators implement the sign-magnitude total order of Table II natively in SFPGT, in SFPSWAP, and — measured here, absent from vendor documentation — in the packer’s threshold unit, which passes +NaN and zeroes −NaN. GPU radix sorts build this order in software via the float→orderable-uint bit-flip trick [25], [26]; IEEE 754-2019 calls it `totalOrder` [27]; on Tensix the *hardware datapath* imposes it, so filtering, bucketing, and the final compare agree on one order with no key transform on the critical path.

B. Packer primitives

Table II rows 3–5 carry the packer findings. The exponent histogram costs nothing to enable but samples 1 datum in 8 in a fixed positional pattern — where the Wormhole-era documentation says per-datum [24] — so it can miss every extreme value: exact-count selectors built on it are dead on arrival (§III), and only a sampled threshold *seed* survives. The threshold filter splits the same way: a free compare-and-zero for $T \geq 0$, unusable for $T < 0$, leaving the 2-issue SFPU filter as the ISA floor for signed data.

C. Dst window layout and SFPSWAP operand order

Reading a running threshold out of the engine’s Dst-resident window (§IV) requires knowing where rank r lives. Full-region Dst dumps against distinct-monotone input, checked word-for-word against torch rank order at $M=512/1024/2048$, fix the law (Table II): column-major over $M/16$ physical rows. An a-priori derivation of this layout was wrong; calibration replaced it. A decision tracer that accumulated $-\infty$ on first bring-up fixed the companion

fact: SFPSWAP with `mod1=1` routes max to VC — the in-tree comment reads backwards; the vendor functional model agrees with the measurement [24]. These two facts make §IV’s chunk-skip rendezvous implementable.

D. Measured floors and the co-issue law

Four floors bound data-dependent designs (Table II, rows 8–9): counting at 2.0000 cyc/vector exactly; the fp32 unpack-to-dest floor at 3.938 cyc/vector — a different quantity from the filter bench’s 3.855 local floor, kept apart deliberately; a rendezvous at 81/101/98 cyc (`tensix_sync / semaphore / Dst-sentinel poll`), provided the SFPU folds lanes first, since raw-lane reads cost 756–773 cyc. §III prices counting designs with exactly these constants. SFLOADMACRO explains why the count rate is a law: `Load+Simple+{MAD|Store}` maps onto three sub-units that co-issue for free (1.000–1.003 cyc/vector), but reductions need ≥ 2 issues because SFPIADD is not macro-schedulable — hence 2 cyc/vector is the counting ceiling. Applied constructively, the same law produced the shipped micro-op wins on the incumbent: merge 1.195/1.221/1.235 $\times$ and step 1.053/1.074/1.090 $\times$ at $M=512/1024/2048$, from paired MATH_ISOLATE arms whose control read 2.000 on every quoted run.

VI. EVALUATION

We defend the headline numbers with five exhibits: the 24-cell competition across five measured arms (Table III), the small-k routing before/after (§VI-B), strong scaling against the cost model (Fig. 2), the chunk-skip A/B (Fig. 3), and eight production call-site scenarios (Table IV); §VI-F closes with the measured gap to the vendor’s published roofline.

A. Harness and Measurement Methodology

All measurements in this paper run on a single Tenstorrent Blackhole p150a with a 13×10 worker grid (130 cores) under exclusive access. Operator-level timings are device-side kernel durations from the Tracy profiler [28], taken at operator granularity so that host dispatch never contaminates a number. One device process runs at a time, each sweep cell in its own subprocess, so program caches, allocator state, and JIT artifacts cannot leak between cells. Kernel-level cycle constants (§V) come from a math-isolated two-point slope over iteration counts, canceling marker overhead; each slope run carries a paired control that must reproduce its known $2.000\times$ ratio or is discarded.

Correctness gates. Every performance cell is gated on bit-exact agreement with a PyTorch golden — never a correlation threshold — so a close-but-wrong kernel cannot post a time. The competition run of Table III recorded 0 wrong results across its 121 gated cells. The surrounding suites hold the same line: a 62-cell numerics contract suite, 154 operator tests, 220/220 routed-path tests, a 36-check adversarial battery, a 2×20 -launch hang battery, and 51/51 (counting) plus 21/21 (RISC scan) bit-exact benchmark cells.

Determinism and provenance. Sweeps are deterministic: fixed seeds, `torch.randn` bf16 stimulus (seed 0 for the

TABLE II
MEASURED TENSIX DATAPATH LAWS FOR SELECTION.

Property	Measured behavior (<i>probe</i>)	Consequence for selection
bf16 canonicalization	NaN (any payload) $\rightarrow$ same-sign Inf; $-0 \rightarrow +0$; $\pm$ subnormal $\rightarrow +0$ (sign dropped); index positions preserved; fp32 passes bit-exactly (<i>identity-op probe</i> ; 62-cell contract suite with JSONL divergence ledger)	reference models must canonicalize first; enables exact (never-PCC) correctness gates
Hardware total order	$-\text{NaN} < -\text{Inf} < \dots < -0 < +0 < \dots < +\text{Inf} < +\text{NaN}$, native in SFPGT, SFPSPWAP, and the packer threshold unit; $-\text{NaN}$ survives the threshold, $-\text{NaN}$ is zeroed (<i>failing $\pm\text{NaN}$ differential test</i>)	the compare order is the radix-key order; no float $\rightarrow$ uint bit-flip map needed
Exponent histogram: sampling	enable cost zero (25.175 $\rightarrow$ 25.104 cyc/tile); samples 1-in-8 positionally ($p \bmod 64 < 8$: exactly 128 increments per 1024-datum tile, format-independent) (38/38 functional + 6/6 perf; marker patterns + mod-8 phase sweep)	exact-count designs dead on arrival; survives only as a sampled threshold seed
Exponent histogram: aliasing and traps	bins alias exponent $\wedge 31$ (32 bins; exp 127 $\equiv$ 159); 8-bit counters saturate at 255; sign-blind; WhichPacker ignored in read modes 6/7 (silent $4\times$ overcount); CLREXPST (1 cyc) does not fence in-flight packs (≈ 39 -count leak) (<i>same bench</i>)	consumers must debias $4\times$, guard saturation, and handle sign upstream
Threshold filter	$T \geq 0$: compare-and-zero during pack at 4.034 cyc/vec vs. the bench's 3.855 local floor ($+4.6\%$); $T < 0$: documented UB, acts as $ T $ rounded up to the next power of two (mantissa ignored) (<i>negative-filter bench</i>)	free pre-filter for non-negative data; signed data falls back to SFPGT(SET_VD)+SFPAND, 2 issues/vec — the ISA floor
Dst window layout	$\text{word}(r) = (r \bmod (M/16)) \cdot 16 + \lfloor r/(M/16) \rfloor$: descending window column-major over $M/16$ physical rows; exhaustive at $M=512/1024/2048$ (<i>full-region Dst dumps vs. distinct-monotone input</i>)	the running-threshold read of §IV's chunk skip lands on the right word
SFPSPWAP operand order	mod1=1 routes max $\rightarrow$ VC, min $\rightarrow$ VD; the in-tree comment reads backwards (<i>decision tracer</i> , $-\infty$ accumulation)	cross-lane max folds accumulate the correct operand
Count / rendezvous floors	count rate 2.0000 cyc/vec exactly; fp32 unpack-to-dest floor 3.938 cyc/vec; rendezvous 81/101/98 cyc (tensix_sync / semaphore / Dst sentinel); no-fold 756–773 cyc; MMIO ≈ 10 cyc/word; PassSync ≥ 25.1 cyc (<i>counting + rendezvous benches</i> , MATH_ISOLATE two-point slope)	sets §III's counting ceiling; fold-on-SFPU before any RISC read is mandatory
SFPLOADMACRO co-issue law	Load+Simple+{MAD Store} maps at 1.000–1.003 cyc/vec (three sub-units co-issue free); reductions need ≥ 2 issues (SFPIADD not macro-schedulable) (<i>paired arms with mutation controls</i>)	counting cannot beat 2 cyc/vec; the law behind the shipped merge/step wins

chunk-skip cells). Every CSV row is provenance-stamped with a build fingerprint and a source-tree hash, so each number is attributable to one binary and one tree state.

Trials and spread. Competition cells report medians of 3 trials; chunk-skip cells report medians of 3 trials $\times$ 5 measured iterations with spread below 0.1% everywhere; the single-core baseline sweeps carry standard deviations in-file (e.g., 9,492,327 $\pm$ 1,391 ns at $N=65,534$). The anchor cell ($k=2048$, $N=65,536$) appears in two independent gated runs — 34.3 μ s in the competition sweep, 34.4 μ s in the P -sweep re-pin; each is quoted from its own run, and the 0.1 μ s disagreement bounds run-to-run drift. We state the gap plainly: all results come from one board on one day, and the 121-cell competition run does not commit per-cell standard deviations — a cross-day, higher-trial repeat is scheduled, not done.

Clock caveat. Cycle $\rightarrow$ microsecond conversions in this paper assume a 1.35 GHz busy clock; the clock under load was not captured (the idle clock measures 800 MHz). Every microsecond figure in this section is a direct Tracy measurement; the caveat applies only where a cycle count is restated in time units, flagged where it occurs.

B. Competition Across Five Measured Arms

Table III sweeps $k \in \{512, 1024, 2048\}$ across $N = 2,048$ –262,144 over five measured arms: *pre* — stock `ttnn.topk` before this work; *now* — stock `ttnn.topk` today, including the $\approx 1.17\times$ replay micro-op win that landed in the stock kernels; *routed* — `ttnn.topk` through the new routing, so the caller changes nothing; *op* — the `topk_large_indices`

operator called directly (its columns include the SFPLOADMACRO paths, worth 5–9%); and a *proxy* arm that wraps the byte-identical stock bitonic kernels in the operator's row-parallel harness and reports per-row single-core wall time. Each speedup chain is named where used and never mixed within one claim.

Four headlines. First, the user-facing anchor: at $k=2048$, $N=65,536$, the *pre $\rightarrow$ routed* chain is 631,506 μ s $\rightarrow$ 63.4 μ s = 9,956 $\times$ — with no call-site change. Second, the direct-op anchor: the same cell measures **34.3 μ s on 32 cores** (8×4 rectangle), an 18,401 $\times$ *pre $\rightarrow$ op* chain. Third, across all 24 cells the *pre $\rightarrow$ op* chain spans **632 $\times$ –51,430 $\times$** and *pre $\rightarrow$ routed* spans 56 $\times$ –22,481 $\times$. Fourth, the 1M-context decode shape $k=512$, $N=262,144$ finishes in **23.3 μ s on 104 cores**. The table keeps its own floor honestly: at $k=2048$, $N=2,048$ op and proxy tie (15.74 vs. 15.73 μ s) — a 2,048-wide row admits no column parallelism; the proxy $\rightarrow$ op ratio grows to 43 $\times$ at $k=512$, $N=262,144$.

The closest internal comparison is a fused attention program embedding a top-k reduction: at its native cell ($k=2048$, $N=65,536$) it measures 24.4 μ s on 129 cores, 1.4 $\times$ below the op's 34.3 μ s. Both caveats matter: that time *includes* the attention work fused around the selection, and it is a one-call-site fused program, not a callable operator; a per-core breakdown was not committed, so we claim no more than the two end-to-end numbers.

Small-k routing. Against the ≈ 137 ns/element single-core factory of Section II-C, the routed path wins 40 $\times$ –423 $\times$ across the measured before/after pairs ($k \in \{8, 32, 64\}$,

TABLE III
COMPETITION ACROSS FIVE MEASURED ARMS, 24 CELLS.

k	N	stock ttnn.topk (ms)		routed	op	proxy [†]	roofline	op gap	pre→routed	pre→op
		pre	now	$\mu s (P)$	$\mu s (P)$	μs	μs			
512	2,048	4.4	3.7	70.4 (4)	5.9 (4)	10.0	0.32	18.4×	63×	742×
	4,096	9.5	8.0	71.6 (8)	7.1 (8)	18.7	0.46	15.6×	132×	1,327×
	8,192	19.6	16.5	74.0 (16)	8.4 (16)	37.1	0.59	14.2×	265×	2,321×
	16,384	39.8	33.6	79.1 (32)	9.7 (32)	71.9	0.73	13.3×	503×	4,098×
	32,768	80.2	67.7	86.4 (64)	11.0 (64)	141.4	0.88	12.6×	928×	7,284×
	65,536	161.6	137.7	91.7 (64)	13.2 (64)	280.6	1.12	11.7×	1,763×	12,279×
	131,072	323.9	275.8	107.4 (88)	17.4 (88)	566.5	1.56	11.1×	3,016×	18,569×
	262,144	648.4	552.3	136.8 (104)	23.3 (104)	1,008.1	2.39	9.8×	4,740×	27,817×
1024	2,048	7.5	6.3	134.2 (2)	9.4 (2)	11.7	0.45	21.1×	56×	795×
	4,096	17.6	14.9	135.9 (4)	11.4 (4)	21.3	0.64	18.0×	130×	1,543×
	8,192	37.8	31.9	138.9 (8)	13.3 (8)	41.5	0.82	16.2×	272×	2,833×
	16,384	78.1	65.9	144.9 (16)	15.2 (16)	79.8	1.01	15.0×	539×	5,145×
	32,768	158.8	133.9	152.7 (32)	17.2 (32)	156.5	1.22	14.1×	1,040×	9,234×
	65,536	320.8	273.1	158.1 (64)	19.7 (64)	309.9	1.56	12.6×	2,029×	16,281×
	131,072	644.2	548.2	173.0 (64)	23.5 (64)	616.7	2.17	10.8×	3,723×	27,461×
	262,144	1,291.0	1,098.6	205.6 (88)	32.1 (88)	1,230.2	3.32	9.7×	6,280×	40,199×
2048	2,048	10.0	8.4	29.5 (130)	15.7 (130)	15.7	0.53	29.9×	338×	632×
	4,096	30.1	25.4	33.1 (2)	19.6 (2)	24.6	0.75	26.2×	909×	1,535×
	8,192	70.3	59.4	38.4 (4)	22.9 (4)	47.7	0.97	23.6×	1,831×	3,074×
	16,384	150.8	127.3	46.3 (8)	26.4 (8)	91.8	1.19	22.2×	3,260×	5,705×
	32,768	311.8	263.3	55.5 (16)	30.2 (16)	180.2	1.43	21.1×	5,614×	10,311×
	65,536	631.5	539.5	63.4 (32)	34.3 [‡] (32)	357.0	1.84	18.7×	9,956×	18,401×
	131,072	1,273.7	1,089.7	79.4 (64)	39.7 (64)	710.5	2.56	15.5×	16,039×	32,092×
	262,144	2,557.6	2,188.1	113.8 (64)	49.7 (64)	1,417.4	3.90	12.8×	22,481×	51,430×

Tracy device kernel durations, medians of 3 trials, every cell bit-exact against a PyTorch golden (0 wrong of 121); arms as defined in §VI-B. Roofline: the vendor’s published model [29]. [†]Byte-identical stock bitonic kernels in the operator’s row-parallel harness; per-row single-core wall time, a proxy rather than a shipping configuration. [‡]A fused attention+top-k program measures 24.4 μs on 129 cores at this cell (caveats in §VI-B).

$W = 5,000\text{--}65,536$): $695 \rightarrow 17.2\mu s$ at $k=32$, $W=5,000$ ($40\times$); $9.50\text{ ms} \rightarrow 38.5\mu s$ at $W=65,534$ ($247\times$); $17.95\text{ ms} \rightarrow 42.4\mu s$ at $k=64$, $W=65,536$ ($423\times$). Stock times are 3-trial means (in-file standard deviations $\leq 4.4\mu s$); routed cells are correctness-gated, multi-core cells bit-stable against the pinned baseline sweep. The same routing removes the $\approx 89\times$ width cliff at the $W < 65,535$ bitonic gate (Section II-C).

C. Strong Scaling Versus the Cost Model

Fig. 2 sweeps the slice count P on two cells, 15 measured points total. At $k=2048$, $N=65,536$ the op falls $183.0 \rightarrow 34.4\mu s$ from $P=2$ to $P=32$; at $k=512$, $N=262,144$ it falls $560.2 \rightarrow 23.3\mu s$ from $P=2$ to $P=104$. Both curves are monotone in the factory’s chosen P — the property the log-tree bought over its serial-chain ancestor (§IV) — and the $k=512$ curve flattens as P approaches the 130-core grid capacity. L1 capacity never bounds P : per-slice fetches run 2–8 KB and the read stream sits 10–100 $\times$ below the op time — the scaling limit is the $\lceil C/P \rceil$ work term itself.

The overlay is §IV’s cost model times its two measured merge-unit constants — no further fitting. The model tracks the $k=2048$ curve within 4.3% at every point and predicts the non-monotone bump at $P=24$: $\lceil 32/24 \rceil = 2$ leaf chunks plus a fifth tree level cost more than $P=16$ ’s identical leaf term with four levels (measured 42.8 vs. 39.0 μs ; model 44.1 vs. 38.6 μs). On the $k=512$ curve the model lands within 0.3% at the factory’s operating point $P=104$ but overestimates the deep-serial tail by up to 33% at $P=2$ — the flat two-units-per-chunk charge is conservative on long streams. Since the

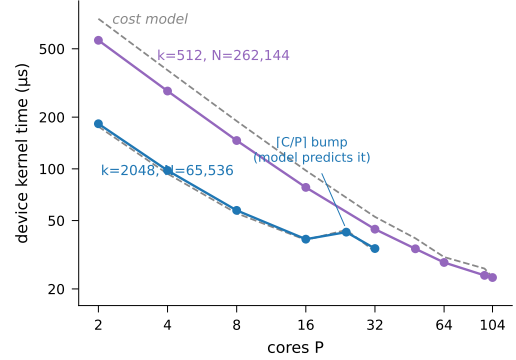


Fig. 2. Runtime versus core count, with cost-model overlay.

factory only ever selects cost-minimal P , the model is accurate where it decides.

D. Chunk-Skip A/B

Fig. 3 measures the row-parallel chunk-skip of §IV on five cells, three arms each: baseline, skip ungated, and skip with the compile-time $K/4$ gate. On the shapes the small- k routing ships to this path, the gated skip wins $279.14 \rightarrow 153.19\mu s$ (-45.1% , $1.82\times$) at rows=2, $k=32$, $N=65,536$, and $279.14 \rightarrow 177.94\mu s$ (-36.3%) at rows=8 — the rows=8 win is smaller because the op time is the slowest of eight i.i.d. rows. On the three cells where the skip law says no win exists, the gate holds the line: $+0.04\%$ and $+0.11\%$ sit within the sub-0.1% measurement spread, and the largest residual, $+0.51\%$ at $k=512$, is the gate-off code path itself

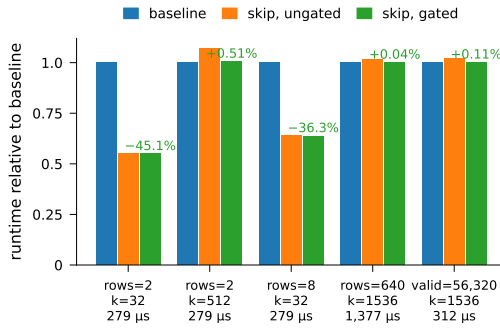


Fig. 3. Chunk-skip runtime relative to baseline.

— loop-code growth on the in-order RISCs, not the skip test. The column-parallel guard cell re-measures at 13.15μs with byte-identical kernels: the tree path is untouched.

These measurements close the pre-build forecast’s loop (§IV-E): the no-go’d column-parallel variant was never built; the redirected row-parallel variant landed inside the predicted regime. One epistemic limit stands: no on-device skip-rate telemetry was captured, so the measured quantity is end-to-end time — the chain runs law → simulated skip → predicted time → measured time.

E. Production Call-Site Scenarios

Table IV replays eight production call-site shapes on the same gated harness — the three consumer classes of the data contract (§II-A) at their real shapes. The wins concentrate where production is single-core today. The Qwen3 shape that opened the paper — the vocabulary shard padded to the one power-of-two width that fails the $W < 65,535$ gate — improves from its 9,596.3μs single-core chain to 217.0μs routed and 193.8μs direct (44–50×); split-vocab sampling on a single chip (deliberately unpadded, $N=64,128$) behaves the same: 8,923.6 → 215.2μs routed, 188.1μs direct.

The table carries its own no-change controls as features. The TP=8 shape already rides the multi-core bitonic: routed measures 1.00× — the honest already-fast row (the direct op still gains 1.4×). The sparse-attention indexer rows already call the op by name — today *is* the op, row-parallel with the chunk skip live — and forcing them through generic routing would cost 20–25%. The MoE gate rows are shapes the routing predicate declines today ($W \geq 4,096$ floor, a measured-region choice); the op — k rounded up to 16 and sliced, the route’s own internal mechanism — measures 4.07μs and 4.04μs against 24.2μs and 77.5μs single-core (5.9×/19.2×); a route extension still needs an A/B against the layout-envelope cost at these shapes.

One caveat is mandatory: the production sampling call sites pass pre-allocated index tensors, sub-core grid pinning, and `stable=True` — each independently disqualifies routing as-is. The routed column is the *canonical* call form with those arguments dropped — a one-line call-site change, not a free win — and the end-to-end tokens/s effect of that change is unmeasured.

F. Gap to the Vendor Roofline

The last two data columns of Table III compare the op against the vendor’s published roofline for this part, including its k -dependent multipliers (0.612/0.850/1.000 for $k=512/1,024/2,048$) [29]. The measured gap spans $9.7\times - 29.9\times$ over all 24 cells and narrows to $9.7\times - 21.1\times$ on the $N \geq 32,768$ cells, shrinking monotonically with N at every k . The known contributors are measured elsewhere: the comparator critical path inside each M -wide leaf pass (§III), the $\lceil \log_2 P \rceil$ tree levels priced by the cost model (§IV), and the unpack floors that tax every datum entering the SFPU (§V). Whether the roofline is attainable at all for comparator-based kernels — or sits below their critical-path floor — is a derivation we do not print here [MISSING-EVIDENCE: roofline-v2 derivation artifact (G6)]; we report the measured gap and leave feasibility open.

VII. RELATED WORK

This section carries the deltas by contribution; inline citations appeared in context.

a) *Mesh selection theory*: Krizanc and Narayanan solved rank selection on mesh-connected processor arrays three decades ago: optimal algorithms on the abstract $n \times n$ mesh [11], a deterministic $1.45n$ -step bound against the $2n+o(n)$ sorting bound [12], and the multi-packet N/p -per-PE regime our $\lceil C/P \rceil$ leaf term inhabits [21]. The theory assumed unit-cost neighbor communication and returned the k -th element alone: no full top- k set with values and indices, no real NoC costs, no atomics-free rendezvous, no measurement. We therefore claim the first *measured, cost-modeled* full top- k on commercial NoC-mesh silicon, with $2\lceil C/P \rceil + \lceil \log_2 P \rceil$ read as the mesh-theory bound instantiated with measured constants (Section IV). Fagin’s threshold algorithm and its distributed descendants [30], [31], [32] merge per-node top- k lists under running thresholds — the same tournament idea at datacenter round-trip costs.

b) *GPU selection*: The GPU lineage runs from Alabi et al. [2] through WarpSelect [18], Dr. Top- k [3], the SC’23 study [17], and RadiK [4], with database-side [33] and row-wise [34] variants; TPU-KNN [9] and two-stage schemes [10] take the approximation escape hatch instead. None of this lineage was ever re-costed on a machine without atomics or scatter — that re-costing is Section III.

c) *Running-threshold pruning*: WarpSelect discards elements below a per-lane running k -th value before they touch its bitonic queues [18] — element-granular, with no analytic skip law. GVR predicts the threshold from the previous decode step and must verify and refine wrong guesses; it reports $1.88\times$ over TensorRT-LLM’s radix select on NVIDIA Blackwell [7]. Chunk-skip (Section IV) skips at chunk granularity, is sound by construction — no verification pass exists to run — and its distribution-free law (Eq. 2) yields a compile-time $K/4$ gate and a pre-build forecast. SpAtten’s quick-select engine [35] and Focus’s streaming concentrator [36] are hardware threshold-engine cousins. The rank statistics are textbook — a running top- k over a random-order stream

TABLE IV
PRODUCTION CALL-SITE SCENARIOS WITH BUILT-IN CONTROLS.

call site	rows	N	k	today μs (P)	routed μs (P)	op μs (P)	today $\rightarrow$ routed	today $\rightarrow$ op
Qwen3 decode sampling, TP=4 (pad)	32	65,536	32	9,596.3 (1)	217.0 (130)	193.8 (130)	44.2 $\times$	49.5 $\times$
decode sampling, TP=8 (pad, pow-2)	32	32,768	32	171.4 (65)	171.3 (65)	121.0 (130)	1.00 $\times$	1.4 $\times$
Llama-3.2 split-vocab sampling	32	64,128	32	8,923.6 (1)	215.2 (130)	188.1 (130)	41.5 $\times$	47.4 $\times$
DeepSeek-V3.2 sparse-attn indexer	160	65,536	2,048	712.5 (130)	892.7 (130)	712.5 (130)	0.80 $\times$	1.00 $\times$
DeepSeek-V4 sparse-attn indexer	160	65,536	512	559.5 (130)	691.1 (130)	559.5 (130)	0.81 $\times$	1.00 $\times$
MiniMax MSA block selection	1	8,192	16	8.4 (16)	116.1 (65)	8.4 (16)	0.07 $\times$	1.00 $\times$
MoE expert gate, top-4 of 128	32	128	4	24.2 (1)	—	4.07 (130)*	—	5.9 $\times$ *
MoE gate fallback, top-10 of 512	32	512	10	77.5 (1)	—	4.04 (130)*	—	19.2 $\times$ *

All rows Tracy-measured, provenance-stamped, iteration counts recorded. “today” is the engine production selects now: single-core linear (rows 1, 3, 7, 8), 65-core stock bitonic (rows 2, 6), or the op itself (rows 4–5, chunk skip live). Op cells include the values output (verified elementwise): $\leq 0.2\%$ over indices-only at sampling shapes, $\approx 6\%$ at the tiny gate shapes. Core counts are launched grids (min(rows, 130) active on the row-parallel path). * k rounds up to 16 (the op’s floor) and slices; the routing predicate declines these shapes today.

updates $k \ln(n/k) + O(k)$ times [37] — but we find no published chunked corollary, compile-time gate, or soundness proof for skipping inside a bitonic cascade.

d) Accelerator characterization: Published Tensix studies characterize performance — stencils on Grayskull [38], matmul [39]; FFT [40], stencils [41], numerical kernels [42], operator fusion [43], and N-body [44] on Wormhole. An informal Blackhole microbenchmark report circulated in 2025, but its link is dead and no archive exists [23] — itself an argument for archival characterization. None covers datapath numerics or selection primitives. Section V’s delta over the software float re-encodings [25], [26] and IEEE 754-2019 totalOrder [27]: the Tensix datapath imposes the total order and the bf16 canonicalization in *hardware*, not software.

e) The class-wide gap: We find no sorting or selection publication for Cerebras WSE, Groq, SambaNova, Occamy [45], Manticore [46], or Esperanto silicon; Graphcore ships `popops::TopK` with no published algorithm or performance study [47]. This paper prices the ingredients any member of the dataflow-mesh class must pay — but fills the gap for only one member.

VIII. METHODOLOGY: AN AGENTIC DESIGN-SPACE CAMPAIGN

An LLM-agent campaign with human review produced these measurements. Prior agentic kernel work optimizes *given* kernels against benchmark suites [48], [49], [50], [51], [52]; this campaign ran a design-space study with measured no-gos on novel silicon — we claim no first-ness for the approach, only its record.

The campaign ran as research $\rightarrow$ implement $\rightarrow$ validate gates with stop rules registered before any measurement — a contract gate (semantics and harness pinned first), a materialization gate (no complete emit path with headroom $\Rightarrow$ stop selector work), and an engine shootout; the radix track died by its own rule (Section III-D).

The most transferable instrument was forecast-before-build (Section IV-E): a calibrated host simulation no-go’d the column-parallel chunk-skip variant — never built — and redirected the build to the row-parallel variant that measured -45.1% ($1.82\times$) on silicon (§VI-D).

Agent-generated numbers earn trust only under adversarial audit: a multi-agent audit re-verified the working notes’ citations (≈ 55 of 60 exact, none fabricated), re-adjudicated disputed figures on fresh silicon, and caught a faster-and-wrong kernel class with schedule-mutation controls — the AI CUDA Engineer’s reward-hacking failure mode [49]. Correctness gates make the audit decidable: every cell fails closed against a bit-exact golden (§VI-A), so wrong-but-close kernels cannot survive. The campaign also surfaced, and reported upstream, two vendor perf-harness bugs: a stimuli configuration never written, and a semaphore leak that deadlocks later tests on an un-reset device.

The process also failed in ordinary ways: the a-priori Dst-layout derivation was wrong until calibration dumps replaced it (§V-C), and one working-note figure was invented outright until the audit deleted it; both are logged in the released notes, and neither reached a measured claim.

IX. CONCLUSION

We measured exact top- k across its design space on a Blackhole p150a and closed a four-orders-of-magnitude gap: $631.5\text{ ms} \rightarrow 63.4\mu\text{s}$ ($9,956\times$) at the anchor shape, $23.3\mu\text{s}$ on 104 cores at the 1M-context shape under the validated cost model, $1.82\times$ more from sound chunk-skipping in the forecast-selected row-parallel regime. The design-space answer: on a NoC mesh without scatter, the winning economics are parallelized comparison networks (A) plus sound rank-statistic skipping — the weak form of (D) — which is Floyd–Rivest’s lesson [19], not radix’s.

The negative result ships with its own expiry conditions — the counting family (B) reopens if any one holds: (1) a device-side compaction or gather primitive appears, or a composed producer+consumer compressed-stream benchmark proves $\geq 2\times$ headroom at $k=2048$, $N \geq 262,144$ — restoring true (D); (2) k outgrows the 2048-element LLK ceiling, ending the additivity of counting passes; (3) the consumer becomes a sampler that never materializes the top- k set; (4) N grows until full-data passes (C) dominate. Future work follows the open gaps: a Wormhole port, energy alongside time, condition (1)’s composed compressed-stream benchmark, and end-to-end tokens/s once call sites drop the disqualifying arguments.

REFERENCES

- [1] J. Vasiljevic and D. Capalija, “Blackhole & TT-Metalium: The standalone AI computer and its programming model,” in *Proc. IEEE Hot Chips 36*, 2024.
- [2] T. Alabi, J. D. Blanchard, B. Gordon, and R. Steinbach, “Fast k-selection algorithms for graphics processing units,” *ACM Journal of Experimental Algorithmics*, vol. 17, 2012.
- [3] A. Gaihre *et al.*, “Dr. top-k: Delegate-centric top-k on GPUs,” in *Proc. SC*, 2021, arXiv:2109.08219.
- [4] “RadiK: Scalable and optimized GPU-parallel radix top-k selection,” in *Proc. ACM ICS*, 2024, arXiv:2501.14336. UNVERIFIED: verify authors.
- [5] A. Fan, M. Lewis, and Y. Dauphin, “Hierarchical neural story generation,” in *ACL*, 2018, introduces top- k sampling for neural text generation. arXiv:1805.04833.
- [6] DeepSeek-AI, “Deepseek-v3.2-exp: Boosting long-context efficiency with deepseek sparse attention,” DeepSeek-AI, Tech. Rep., 2025, sparse-attention indexer selecting top-2048 keys per layer. [verify exact title/identifier before camera-ready].
- [7] “Guess-verify-refine: Data-aware top-k for sparse-attention decoding on Blackwell via temporal correlation,” *arXiv preprint*, 2026, arXiv:2604.22312. UNVERIFIED: verify authors.
- [8] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, “Outrageously large neural networks: The sparsely-gated mixture-of-experts layer,” in *ICLR*, 2017, noisy top- k expert gating. arXiv:1701.06538.
- [9] F. Chern, B. Hechtman, A. Davis, R. Guo, D. Majnemer, and S. Kumar, “TPU-KNN: K nearest neighbor search at peak FLOPs,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022, uNVERIFIED: verify author list.
- [10] “A faster generalized two-stage approximate top-k,” *arXiv preprint*, 2025, arXiv:2506.04165; OpenReview iqZ1Crpjz. UNVERIFIED: verify authors.
- [11] D. Krizanc and L. Narayanan, “Optimal algorithms for selection on a mesh-connected processor array,” in *Proc. IEEE Symposium on Parallel and Distributed Processing (SPDP)*, 1992.
- [12] “Fast deterministic selection on mesh-connected processor arrays,” *Algorithmica*, 1.45n steps on an $n \times n$ mesh vs $2n + o(n)$ sort-based; earlier version LNCS, doi 10.1007/3-540-54967-6_79. UNVERIFIED: verify authors/year.
- [13] Intel Corporation, “x86-simd-sort: SIMD-accelerated sorting and partial sorting,” <https://github.com/intel/x86-simd-sort>, 2023, uNVERIFIED: verify release/year before camera-ready.
- [14] M. Blacher, J. Giesen, P. Sanders, and J. Wassenberg, “Vectorized and performance-portable Quicksort,” *Software: Practice and Experience*, 2023, arXiv:2205.05982. UNVERIFIED: verify authors/venue/volume before camera-ready.
- [15] Tenstorrent, “TT-NN operator reference: `ttnn.topk`,” TT-NN documentation, 2026, accessed 2026-08-16.
- [16] —, “`topk_local_sort` bitonic top-k kernel discussion,” tt-metal issue tracker, #33492, engineering artifact; documents the bitonic local-sort top-k path. [verify year/permalink before camera-ready].
- [17] “Parallel top-k algorithms on GPU: A comprehensive study and new methods,” in *Proc. SC*, 2023, uNVERIFIED: verify authors before camera-ready.
- [18] J. Johnson, M. Douze, and H. Jégou, “Billion-scale similarity search with GPUs,” *IEEE Transactions on Big Data*, vol. 7, no. 3, 2019, warpSelect. arXiv:1702.08734.
- [19] R. W. Floyd and R. L. Rivest, “Expected time bounds for selection,” *Communications of the ACM*, vol. 18, no. 3, pp. 165–172, 1975.
- [20] N. Samardzic, W. Qiao, V. Aggarwal, M.-C. F. Chang, and J. Cong, “Bonsai: High-performance adaptive merge tree sorting,” in *Proc. ACM/IEEE ISCA*, 2020.
- [21] “Multi-packet selection on mesh-connected processor arrays,” in *Proc. International Parallel Processing Symposium (IPPS)*, 1992, uNVERIFIED: verify authors.
- [22] Z. Jia, B. Tillman, M. Maggioni, and D. P. Scarpazza, “Dissecting the Graphcore IPU architecture via microbenchmarking,” Citadel, Tech. Rep., 2019, arXiv:1912.03413.
- [23] “Dissecting the Tenstorrent Blackhole architecture via microbenchmarking,” Informal technical report, Tech. Rep., 2025, uNVERIFIED: formerly at aspl0s.dev (404 as of 2026-08-16); locate archived copy before submission.
- [24] Tenstorrent, “Tensix ISA documentation (Blackhole and Wormhole),” Vendor ISA documentation, 2025, `Dst`, `SFPGT`, `SFSPSWAP`, `SFPL0ADMACRO`, and packer exponent-histogram sections; the Wormhole packer-histogram page documents per-datum counting, contradicted on Blackhole by the measurements of §5 (1-in-8 positional sampling).
- [25] M. Herf, “Radix tricks,” <http://stereopsis.com/radix.html>, 2001.
- [26] D. Merrill and A. Grimshaw, “High performance and scalable radix sorting,” *Parallel Processing Letters*, vol. 21, no. 2, 2011.
- [27] IEEE, “IEEE standard for floating-point arithmetic (IEEE 754-2019),” IEEE Std 754-2019, 2019, totalOrder predicate, §5.10.
- [28] B. Taudul, “Tracy: A real-time, nanosecond-resolution frame profiler,” <https://github.com/wolfpld/tracy>, 2024, device-side timeline capture as integrated in the vendor’s Metalium runtime.
- [29] Tenstorrent, “LLM performance model: operator roofline tables,” Tenstorrent `llm_perf` performance-model repository, 2026, roofline tables for Blackhole with top- k multipliers 0.612/0.850/1.000 for $k=512/1,024/2,048$. UNVERIFIED: pin exact revision before camera-ready.
- [30] R. Fagin, A. Lotem, and M. Naor, “Optimal aggregation algorithms for middleware,” *Journal of Computer and System Sciences*, vol. 66, no. 4, 2003.
- [31] P. Cao and Z. Wang, “Efficient top-k query calculation in distributed networks,” in *Proc. ACM PODC*, 2004.
- [32] S. Michel, P. Triantafillou, and G. Weikum, “KLEE: A framework for distributed top-k query algorithms,” in *Proc. VLDB*, 2005. [Online]. Available: <https://www.vldb.org/archives/website/2005/program/paper/thu/p637-michel.pdf>
- [33] A. Shanbhag, H. Pirk, and S. Madden, “Efficient top-k query processing on massively parallel hardware,” in *Proc. ACM SIGMOD*, 2018.
- [34] “RTop-K: Ultra-fast row-wise top-k selection for neural network acceleration on GPUs,” *arXiv preprint*, 2024, arXiv:2409.00822. UNVERIFIED: verify authors/venue.
- [35] H. Wang, Z. Zhang, and S. Han, “SpAtten: Efficient sparse attention architecture with cascade token and head pruning,” in *Proc. IEEE HPCA*, 2021, arXiv:2012.09852.
- [36] “Focus: A streaming concentration architecture for efficient vision-language models,” *arXiv preprint*, 2025, arXiv:2512.14661. UNVERIFIED: verify authors; read full text (closest architecture-paper analog to a chunked running-threshold engine).
- [37] D. E. Knuth, *The Art of Computer Programming, Volume 3: Sorting and Searching*, 2nd ed. Addison-Wesley, 1998.
- [38] N. Brown and R. Barton, “Accelerating stencils on the Tenstorrent Grayskull RISC-V accelerator,” in *Proc. SC Workshops*, 2024, arXiv:2409.18835.
- [39] “Assessing Tenstorrent’s RISC-V MatMul acceleration capabilities,” *arXiv preprint*, 2025, arXiv:2505.06085. UNVERIFIED: verify authors.
- [40] N. Brown, J. Davies, and F. Le Clair, “Exploring fast Fourier transforms on the Tenstorrent Wormhole,” in *Proc. ISC High Performance Workshops*, 2025, arXiv:2506.15437. UNVERIFIED: verify given names.
- [41] “Stencil computations on Tenstorrent Wormhole,” *arXiv preprint*, 2026, arXiv:2605.07599. UNVERIFIED: verify authors.
- [42] “Numerical kernels on a spatial accelerator: A study of Tenstorrent Wormhole,” *arXiv preprint*, 2026, arXiv:2603.23343. UNVERIFIED: verify authors.
- [43] “Operator fusion for LLM inference on the Tensix architecture,” *arXiv preprint*, 2026, arXiv:2606.09879. UNVERIFIED: verify authors.
- [44] Amati *et al.*, “Accelerating gravitational n-body simulations using the RISC-V-based Tenstorrent Wormhole,” in *Proc. SC Workshops*, 2025, arXiv:2605.02744 / 2509.19294. UNVERIFIED: verify full author list.
- [45] “Occamy: A 432-core dual-chiplet dual-HBM2E 768-DP-GFLOP/s RISC-V system for 8-to-64-bit dense and sparse computing in 12nm FinFET,” *arXiv preprint*, 2025, arXiv:2501.07330. UNVERIFIED: verify authors.
- [46] F. Zaruba, F. Schuiki, and L. Benini, “Manticore: A 4096-core RISC-V chiplet architecture for ultraefficient floating-point computing,” *IEEE Micro*, vol. 41, no. 2, 2021.
- [47] Graphcore, “TopK — Poplar and PopLibs API reference (popops), v3.1.0,” <https://docs.graphcore.ai/projects/poplar-api/en/3.1.0/poplibs/popops/TopK.html>, 2023.
- [48] A. Ouyang *et al.*, “KernelBench: Can LLMs write efficient GPU kernels?” *arXiv preprint*, 2025, arXiv:2502.10517. UNVERIFIED: verify author list.
- [49] Sakana AI, “The AI CUDA engineer,” Sakana AI, Tech. Rep., 2025, includes the reward-hacking cautionary tale motivating verification discipline.

- [50] “AccelOpt: A self-improving LLM agentic system for AI accelerator kernel optimization,” *arXiv preprint*, 2025, arXiv:2511.15915. UNVERIFIED: verify authors.
- [51] “KForge: Program synthesis for diverse AI hardware accelerators,” *arXiv preprint*, 2025, arXiv:2511.13274; see also arXiv:2606.02963. UNVERIFIED: verify authors; check target list for Tenstorrent.
- [52] “PEAK: A performance engineering AI-assistant for GPU kernels powered by natural language transformations,” *arXiv preprint*, 2025, arXiv:2512.19018. UNVERIFIED: verify authors.